AIMLCZG546 - Software Engineering for Machine Learning

Assignment II

SOFTWARE ENGINEERING FOR MACHINE LEARNING:

PRODUCTION-READY AI RESUME SCREENING AND JOB ROLE PREDICTION SYSTEM

Group Details

Group No: 25

Group Member Names with Contribution

Sl. No	BITS ID	Name	Contribution of Team Member (Qualitative)	Percentage Contribution
1	**2024AC05325**	**Haridass K**	Machine learning model development, production code implementation, FastAPI REST API development, Streamlit application development, automated testing, documentation preparation and review, final project integration, and report preparation.	**100%**
2	**2024AC05104**	**Sathish T**	Requirements analysis, production code implementation support, REST API validation, automated testing support, application validation, documentation preparation and review, and report preparation support.	**100%**
3	**2024AC05651**	**Tejaal M**	Data preparation and preprocessing, model evaluation, quality metrics implementation and validation, automated testing support, application validation, documentation preparation and review, and report preparation support.	**100%**
4	**2024AC05728**	**Sanjayan S**	Streamlit UI enhancement, security validation and logging verification, architecture diagrams, application testing, deployment experimentation validation, documentation preparation and review, and report preparation support.	**100%**

Assignment Requirement Traceability Matrix

Assignment Requirement	Report Section
Refactoring and Modular Software Design	Section 5.1
Research Code vs Production Code	Section 5.2
Logging and Error Handling	Section 5.3
Code Formatting (Black, isort, Flake8)	Section 5.4
REST API Implementation	Section 5.5
Automated Testing	Section 6.1
Machine Learning Component Testing	Section 6.2
Model Quality Metrics	Section 6.3
Data Quality Metrics	Section 6.4
Production Experimentation	Section 6.5
Security Consideration	Section 6.6
Streamlit Application	Section 7
Results and Discussion	Section 8
Conclusion	Section 9
Reproducibility	Section 10

1. Introduction

Machine Learning (ML) has become an important technology for automating decision-making across various application domains. However, building an accurate machine learning model alone is not sufficient for real-world deployment. A production-ready ML system requires software engineering practices such as modular software design, testing, logging, security validation, and quality assurance.

This project presents a **Production-Ready AI Resume Screening and Job Role Prediction System** developed as part of the Software Engineering for Machine Learning (SE4ML) course. The application automatically classifies resumes into predefined job categories using Natural Language Processing (NLP) and a Random Forest classifier. In addition to machine learning, the system incorporates a modular architecture, FastAPI REST API, Streamlit user interface, automated testing, logging, security validation, quality metrics, and production experimentation techniques to demonstrate a maintainable and production-oriented ML application.

2. Problem Statement

Organizations receive a large number of resumes for job openings, making manual resume screening time-consuming, inconsistent, and prone to human error. This project addresses this challenge by developing a machine learning-based Resume Screening and Job Role Prediction System that automatically classifies resumes into predefined job categories. The system also incorporates production-oriented software engineering practices, including modular architecture, REST API development, automated testing, logging, security validation, and quality assurance, to improve maintainability, reliability, and scalability.

3. Requirement Specifications and Measurable Goals

Functional Requirements:

- The system shall accept resume text as input.
- The system shall clean and preprocess resume text.
- The system shall predict the suitable job category.
- The system shall display prediction confidence.
- The system shall store prediction history.

Non-Functional Requirements:

- The system should provide predictions within a few seconds.
- The interface should be simple for HR users.
- The system should be maintainable and modular.

Measurable Goals:

- Achieve model accuracy greater than 70%.
- Generate predictions within 5 seconds.
- Store all prediction results in SQLite.

4. Dataset Description

The project uses the **Resume Dataset** obtained from **Kaggle**, containing resumes from multiple professional domains. The dataset is used to train and evaluate a machine learning model for multi-class resume classification.

Dataset Attributes

Column	Description
ID	Unique identifier for each resume
Resume_str	Resume text used for machine learning
Resume_html	HTML representation of the resume
Category	Actual job category (Target Variable)

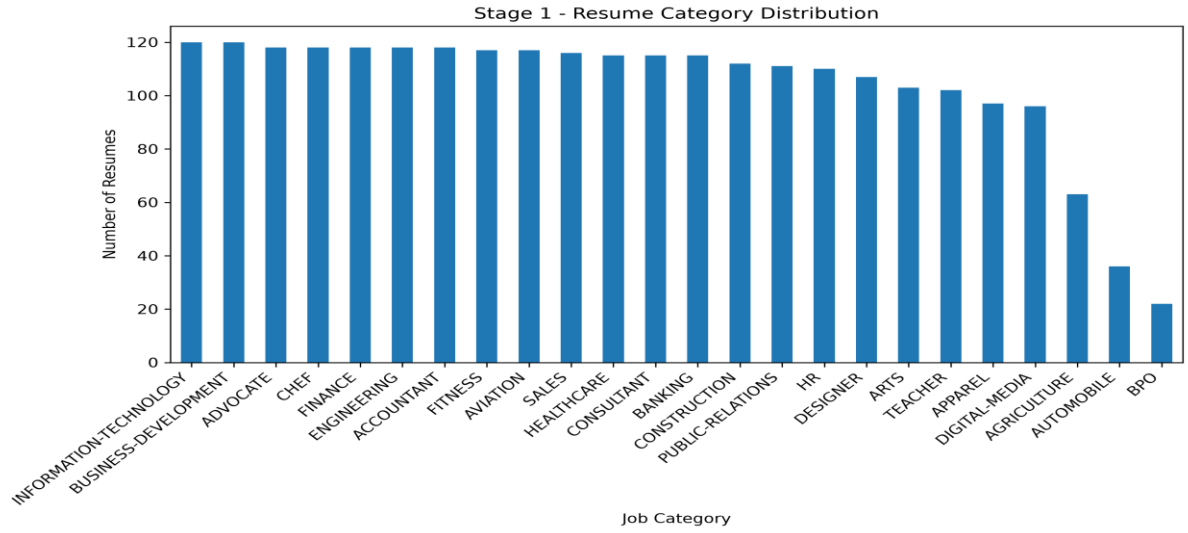
The **Resume_str** column was used as the input feature, while the **Category** column was used as the target label for training and evaluating the machine learning models.

Figure 1:Resume Dataset Overview

Stage 1 - Dataset Overview

Metric	Value
Dataset Shape	(2484, 4)
Number of Columns	4
Missing Values	0
Duplicate Records	0
Unique Job Categories	24

Figure 2:Resume Category Distribution



5. Objective 1: Implementation and Code Sharing

This section presents the software engineering practices implemented to transform the research prototype into a production-ready machine learning application. The project adopts a modular software architecture with separate components for machine learning, API services, security, database operations, and user interface development. Additional production features such as logging, error handling, code quality tools, and REST API implementation were incorporated to improve maintainability, reliability, and scalability.

Figure 3:Overall System Workflow of the AI Resume Screening System

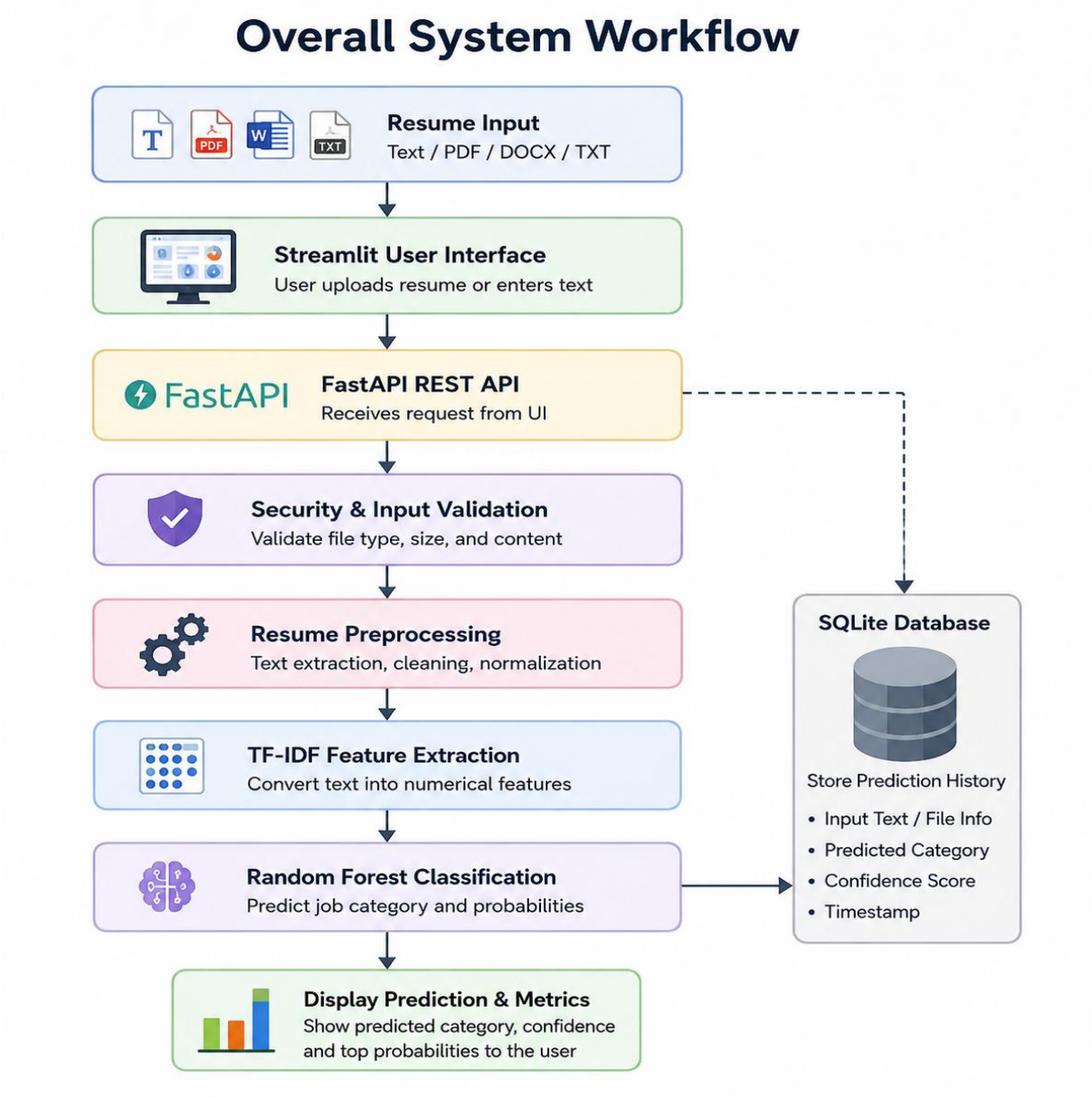
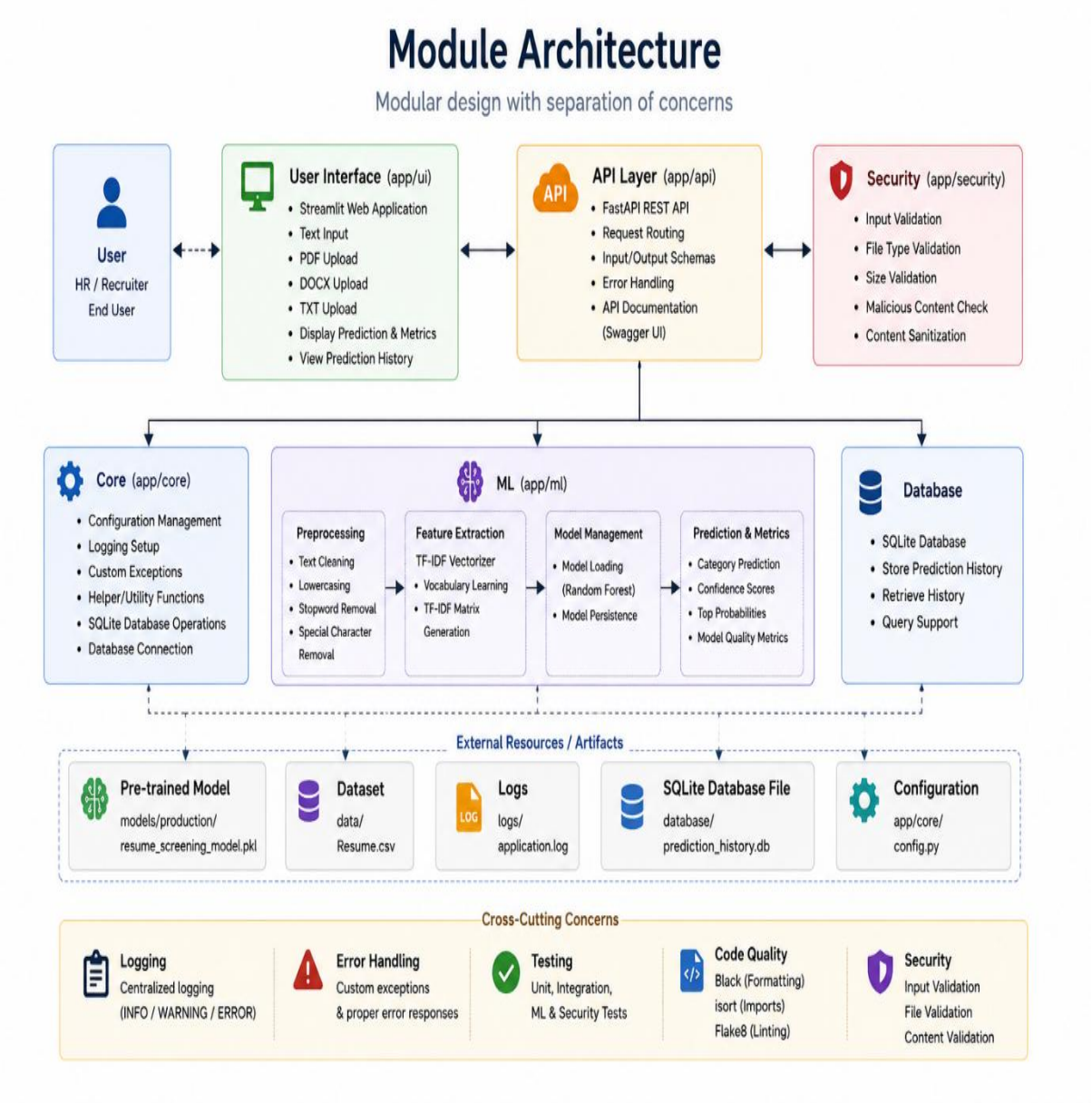


Figure 4:Modular Architecture of the Production Application

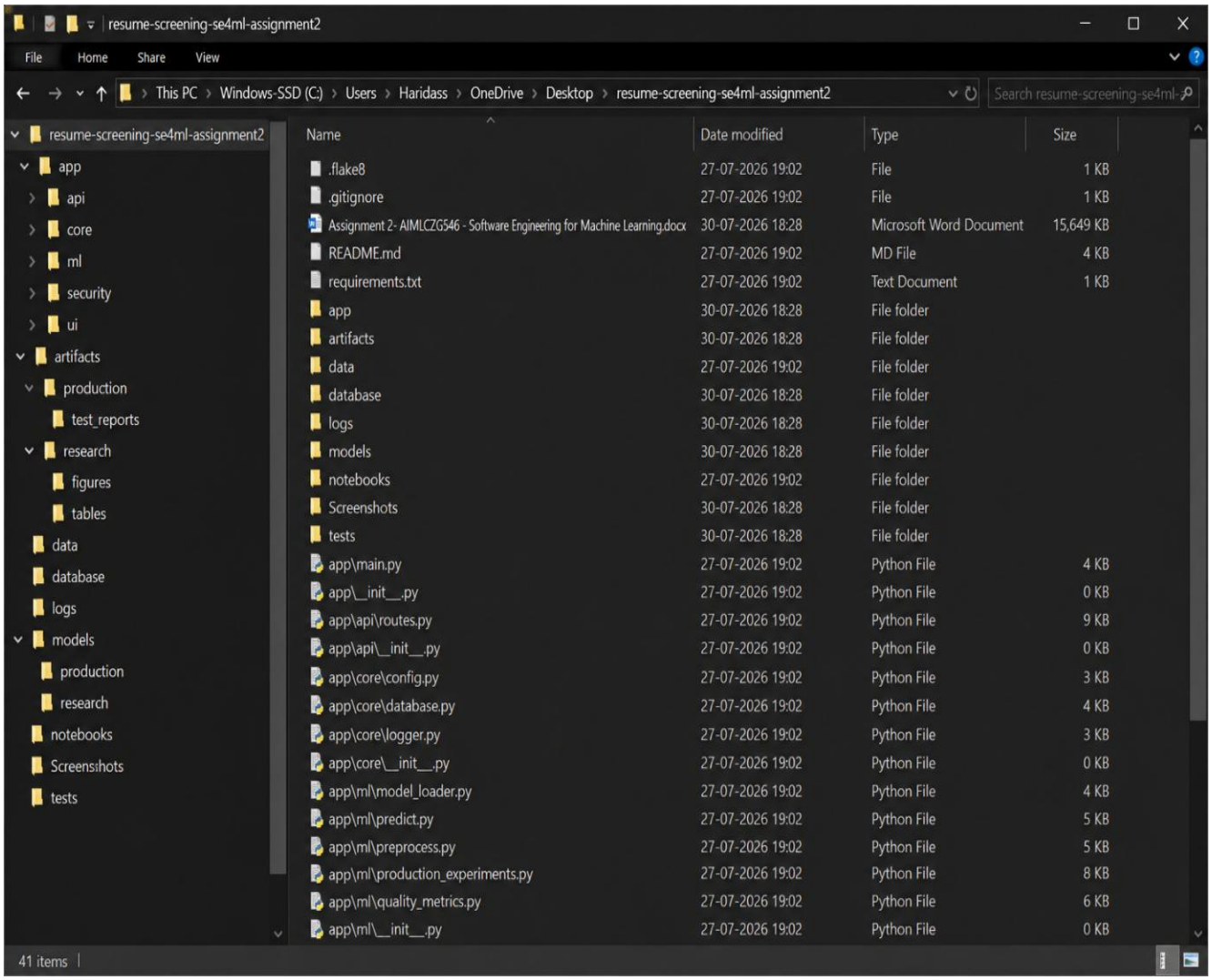


5.1 Refactoring and Modular Software Design

The research notebook was refactored into a modular production application by separating the implementation into independent modules. Each module performs a specific responsibility, making the application easier to maintain, test, and extend.

The project structure includes dedicated modules for machine learning, REST API, security validation, database management, and the Streamlit user interface.

Figure 5. Production Application Project Directory Structure



5.2 Research Code vs Production Code

The project clearly separates the research implementation from the production implementation.

Research Code	Production Code
Jupyter Notebook for experimentation	Modular Python package
Model training and comparison	Production inference pipeline
Interactive analysis	FastAPI REST API
Experimental workflow	Streamlit application
Research artifacts	Production-ready modules

The research notebook was used for data preprocessing, feature engineering, model training, and model comparison. The selected Random Forest model was then integrated into the production application, which provides modular code organization, REST API services, automated testing, logging, and security validation.

Figure 6:Research Notebook Structure

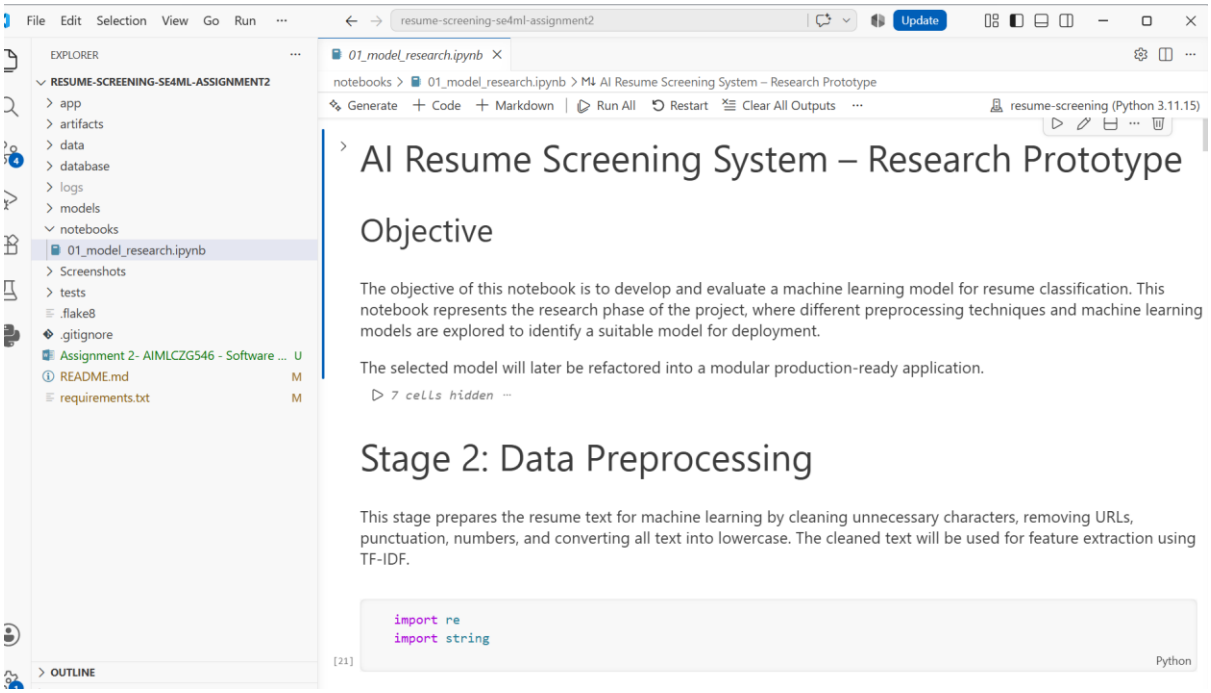
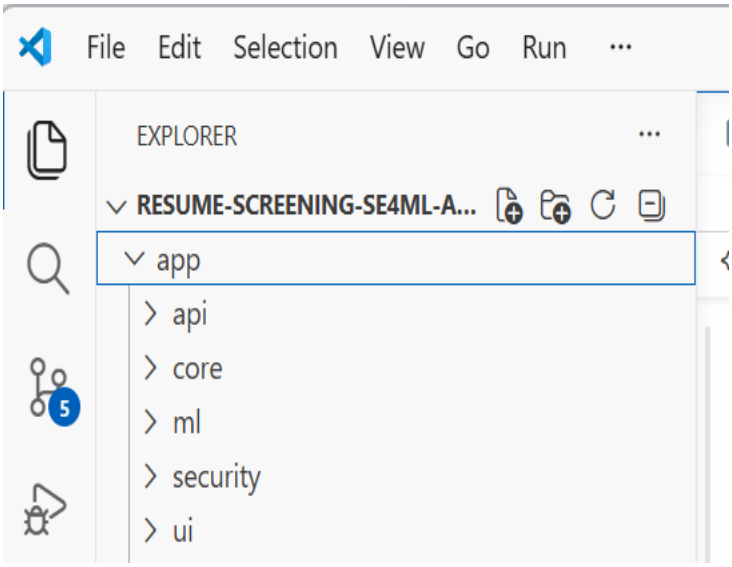


Figure 7:Production Application Folder Structure



5.3 Error Handling and Logging

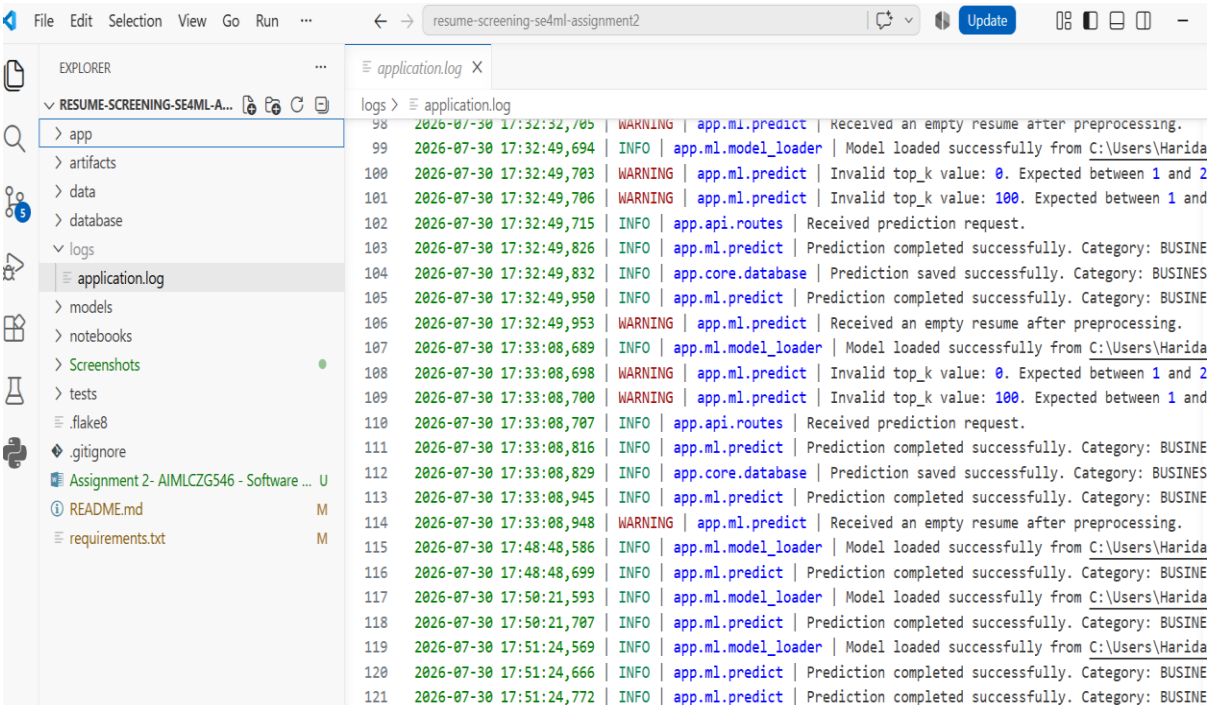
Proper error handling and logging are essential for developing reliable and maintainable machine learning applications. In this project, Python's logging module was used to record application events and system activities at different log levels. Logging was implemented across multiple modules, including the FastAPI application, prediction service, and database operations.

The application uses three primary log levels:

- **INFO** – Records normal application events such as application startup, database initialization, and successful prediction requests.
- **WARNING** – Records invalid user inputs and failed validation checks without interrupting the application.
- **ERROR** – Records unexpected exceptions and internal application failures to assist debugging and troubleshooting.

This implementation improves system monitoring, simplifies debugging, and provides traceability of application events during execution.

Figure 8:Application Logging and Error Handling



5.4 Code Formatting and Linting

To improve code readability and maintain consistency, the project applies automated code formatting and linting tools. These tools ensure that the codebase follows standard Python coding conventions and remains easy to maintain. The following tools were used:

- **Black** – Automatic code formatting.
- **isort** – Automatic organization of Python import statements.
- **Flake8** – Static code analysis and style compliance checking.

All source code was successfully formatted and verified before project completion.

Figure 9. Flake8 Static Code Analysis Before Code Quality Fixes

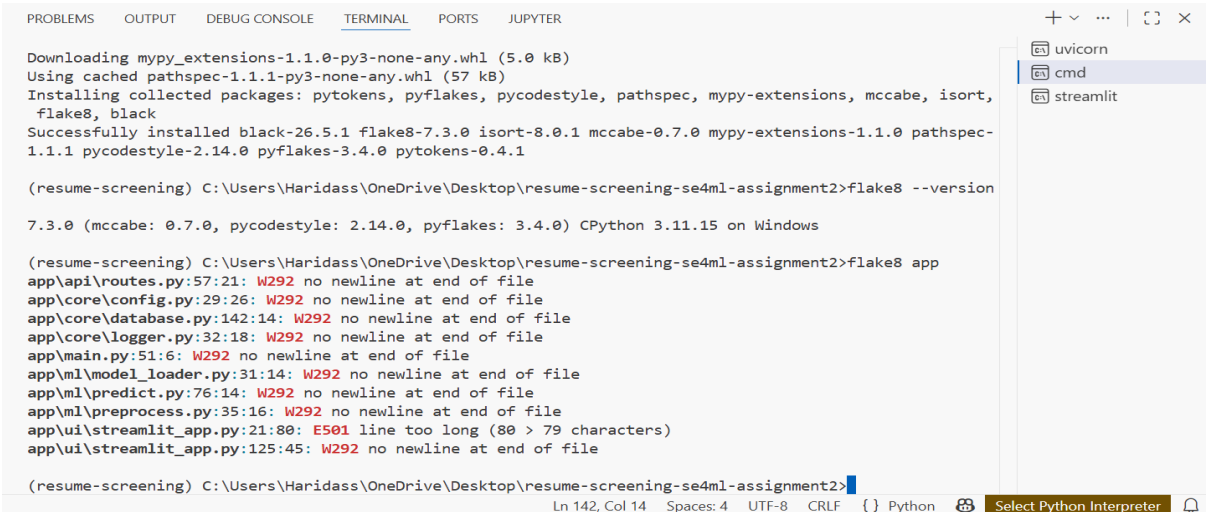


Figure 10. Code Formatting Using Black and isort

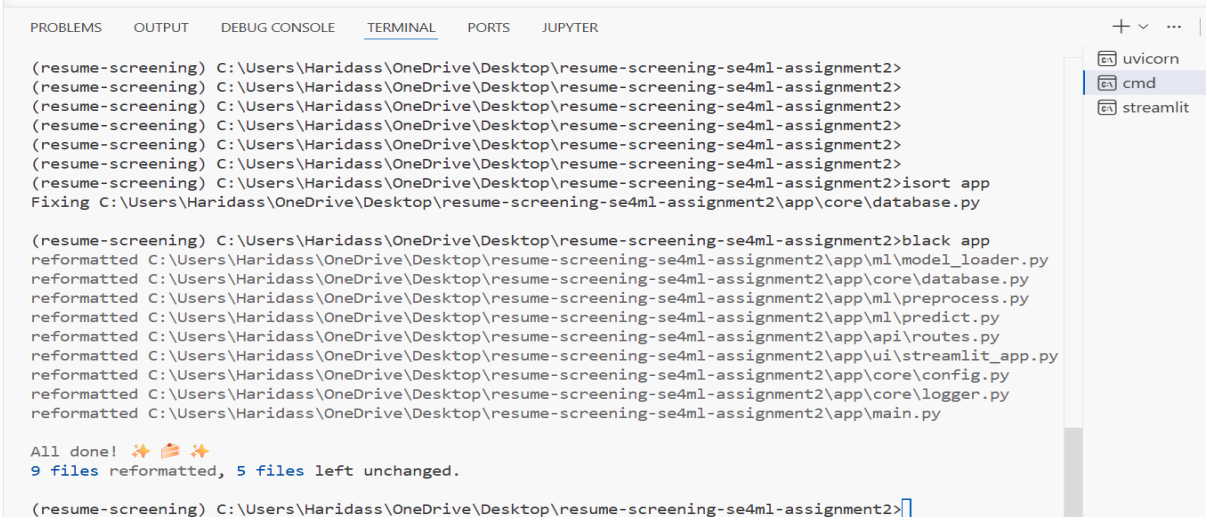
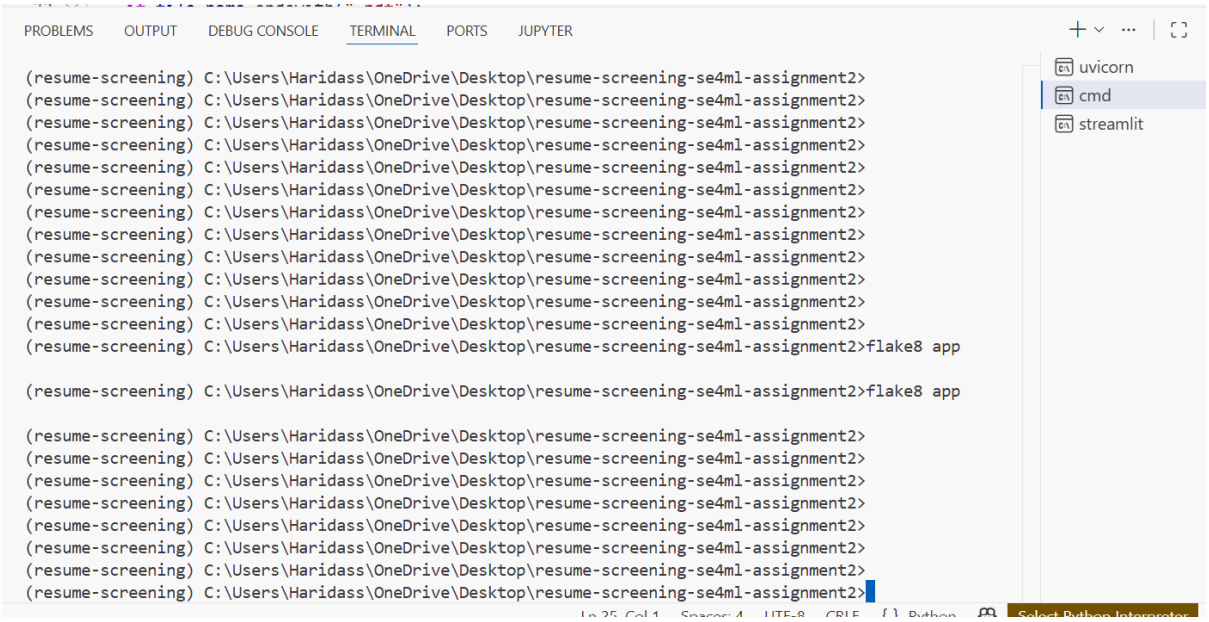


Figure 11. Flake8 Static Code Analysis After Code Quality Fixes



5.5 REST API Implementation

A RESTful API was developed using **FastAPI** to expose the machine learning model for resume classification. The API provides a standardized interface for submitting resume text and receiving prediction results in JSON format. FastAPI was selected because of its high performance, automatic API documentation, and built-in request validation.

The application implements clear request and response schemas using **Pydantic** models, along with appropriate HTTP status codes and exception handling to ensure reliable communication between the client and the machine learning model.

The implemented API endpoints are:

Endpoint	Method	Description
/	GET	Returns the API welcome message.
/health	GET	Checks the health status of the application.
/predict	POST	Predicts the job category for the submitted resume text.

Figure 12. FastAPI Swagger API Overview

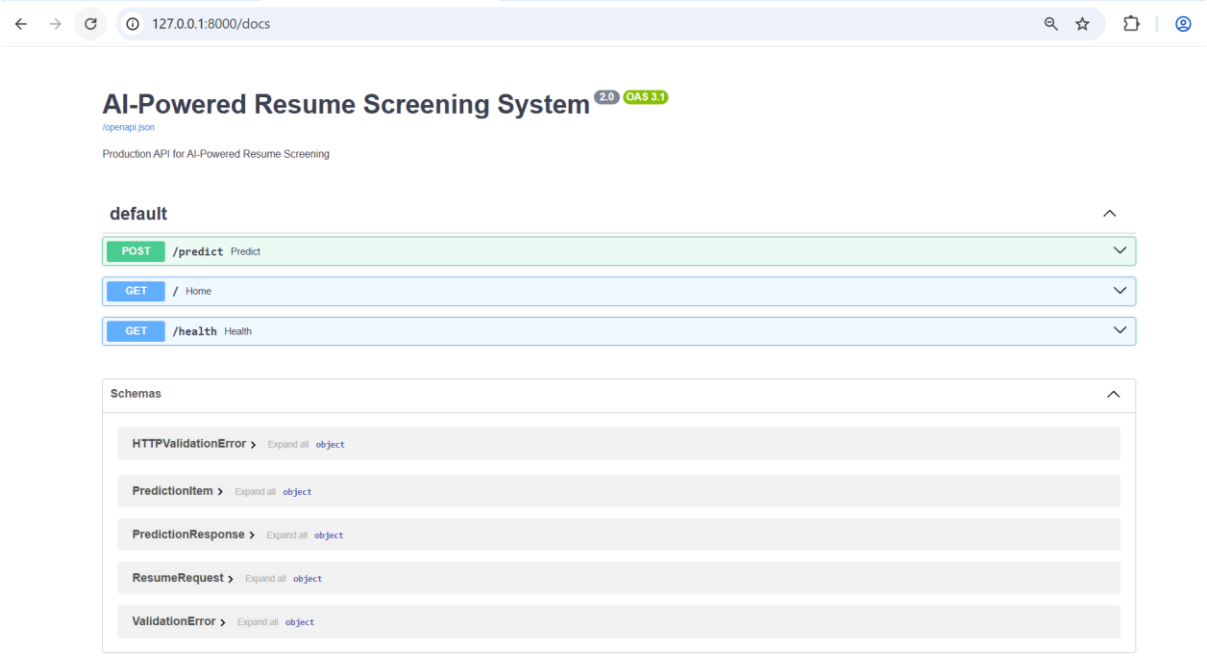


Figure 13. FastAPI Swagger Response Schema and HTTP Status Codes

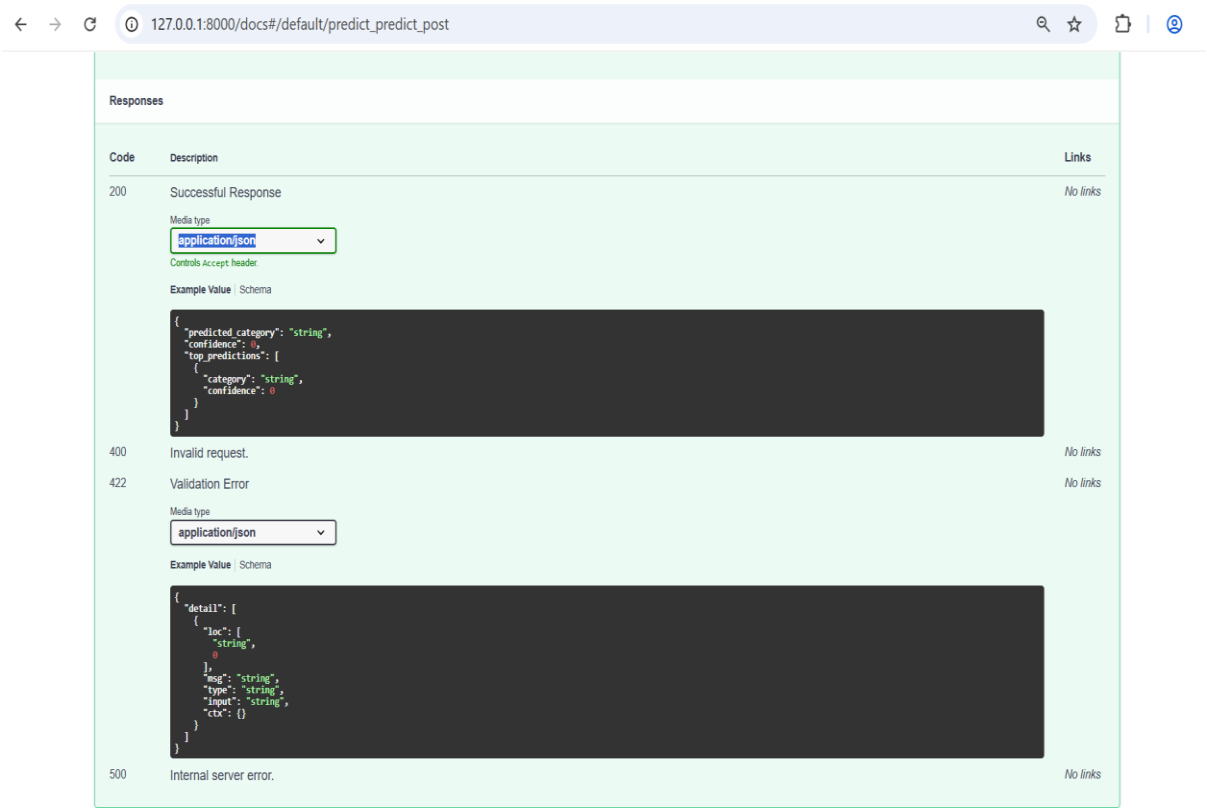
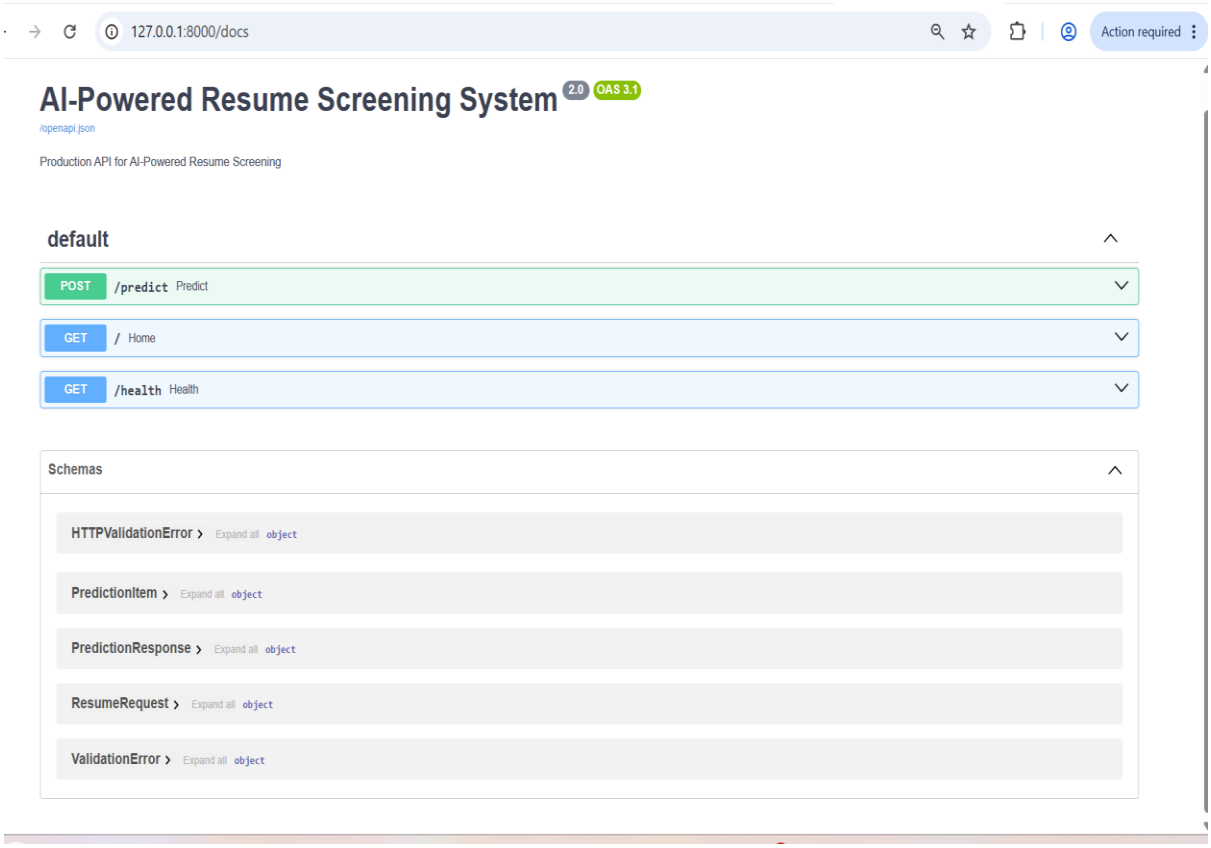


Figure 14. FastAPI Swagger API Documentation and Data Schemas



6. Objective 2: Quality Assurance

This section presents the quality assurance practices implemented to improve the reliability and correctness of the machine learning application. Multiple testing techniques, machine learning component validation, model and data quality evaluation, production experimentation, and security validation were incorporated to verify the functionality and robustness of the system.

6.1 Automated Testing

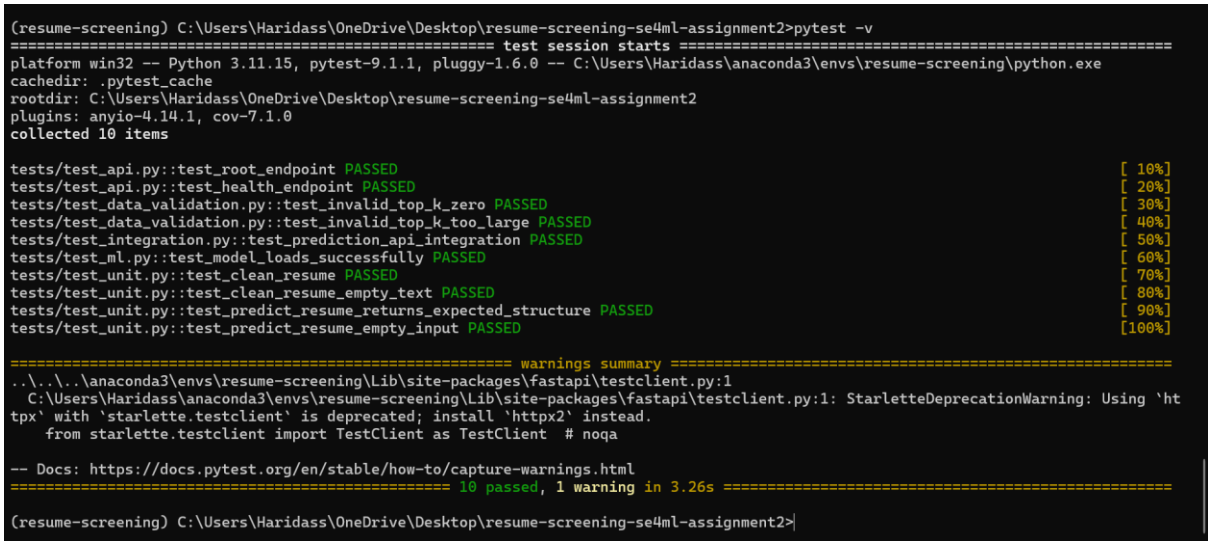
Automated testing was performed using the **pytest** framework to verify the correctness of the application. Different types of tests were implemented to validate individual components as well as the overall system functionality. The implemented tests include:

- **Unit Tests** – Validate individual functions such as preprocessing and prediction.
- **Integration Tests** – Verify the interaction between application modules.
- **API Tests** – Validate FastAPI endpoints and responses.

- **Data Validation Tests** – Verify input validation and data integrity.

A total of **10 automated tests** were successfully executed, confirming the correctness and stability of the application.

Figure 15:Automated Testing Results



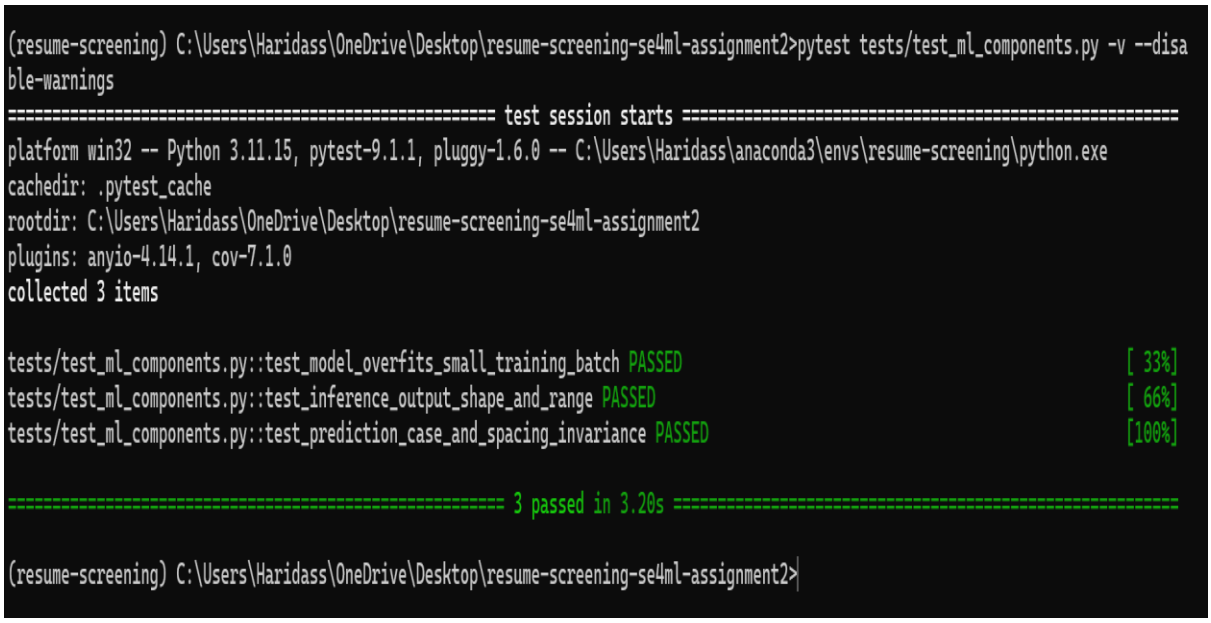
6.2 Machine Learning Component Testing

In addition to software testing, specific tests were implemented to validate the machine learning components of the application. These tests verify that the model behaves as expected during training and inference, ensuring reliable prediction performance. The implemented machine learning component tests include:

- **Model Training Test** – Verifies that the model can successfully overfit a small training dataset.
- **Inference Output Test** – Validates the prediction output format and confidence score range.
- **Prediction Invariance Test** – Confirms that predictions remain consistent for equivalent resume text with different letter casing and spacing.

All machine learning component tests were successfully executed, confirming the correctness and robustness of the prediction pipeline.

Figure 16:Machine Learning Component Testing Results



6.3 Model Quality Metrics

The performance of the selected machine learning model was evaluated using standard classification metrics to assess its prediction accuracy and overall reliability. These metrics provide a quantitative measure of the model's effectiveness on the evaluation dataset.

The model quality metrics evaluated include:

- **Accuracy** – Measures the overall prediction accuracy of the classifier.
- **Weighted F1-Score** – Evaluates the balance between precision and recall across all job categories.
- **Weighted Precision** – Measures the proportion of correctly predicted positive classifications.
- **Weighted Recall** – Measures the proportion of actual positive classifications correctly identified by the model.

The evaluated metrics indicate that the selected model provides reliable and consistent performance for automated resume classification.

Figure 17:Model Quality Metrics

```
(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>python -m app.ml.quality_metrics

Model Quality Metrics
----------------------------------------
Accuracy: 0.7505 (75.05%)
Weighted Precision: 0.7689 (76.89%)
Weighted Recall: 0.7505 (75.05%)
Weighted F1-Score: 0.7316 (73.16%)

Results saved to: C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2\artifacts\production\model_quality_metrics.cs
v

(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>python -m app.ml.quality_metrics

Model Quality Metrics
----------------------------------------
Accuracy: 0.7505 (75.05%)
Weighted Precision: 0.7689 (76.89%)
Weighted Recall: 0.7505 (75.05%)
Weighted F1-Score: 0.7316 (73.16%)

Results saved to: C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2\artifacts\production\model_quality_metrics.cs
v

Data Quality Metrics
----------------------------------------
Missing Value Rate: 0.0000 (0.00%)
Duplicate Record Rate: 0.0008 (0.08%)
Schema Validation Rate: 0.9996 (99.96%)
Class Imbalance Ratio: 5.45

Results saved to: C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2\artifacts\production\data_quality_metrics.csv

(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>
```

6.4 Data Quality Metrics

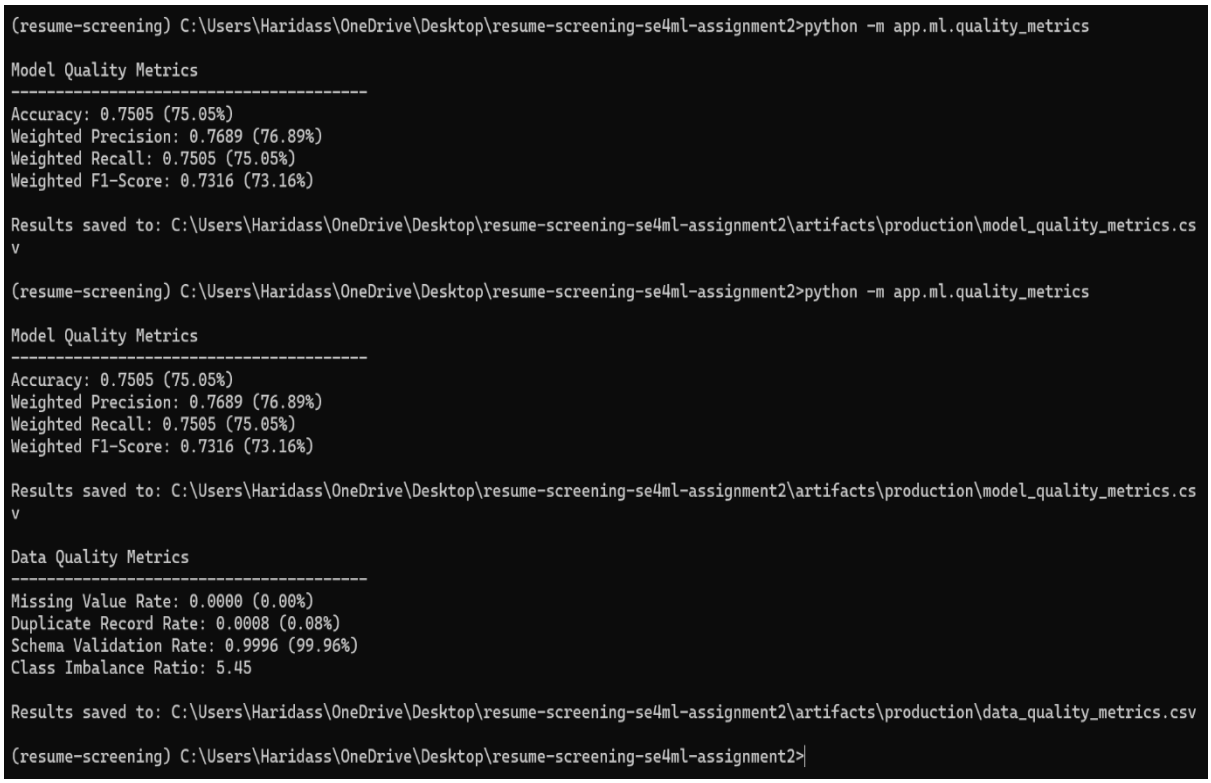
Data quality assessment was performed to ensure that the dataset used for model training was complete, valid, and suitable for machine learning. Evaluating data quality helps identify potential issues that may affect model performance and reliability.

The following data quality metrics were evaluated:

- **Schema Validation** – Verifies that the dataset contains the expected columns and data types.
- **Missing Value Check** – Confirms that the dataset does not contain missing or null values in the required fields.

The data quality evaluation confirmed that the dataset satisfied the required schema and contained no missing values, making it suitable for model training and inference.

Figure 18: Data Quality Metrics



6.5 Production Experimentation

To demonstrate production-oriented deployment strategies, three commonly used experimentation approaches were implemented and evaluated. These approaches help validate new model versions while minimizing deployment risk.

The implemented production experimentation techniques include:

- **Shadow Deployment** – Executes the new model alongside the production model without affecting end users.
- **Canary Release** – Gradually exposes the new model to a subset of users before full deployment.
- **A/B Testing** – Compares two model versions using separate user groups to evaluate performance.

These approaches demonstrate how machine learning models can be validated safely before full-scale production deployment.

Figure 19:Shadow Deployment Experiment Results

```
(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>python -m app.ml.production_experiments

Canary Release Results
----------------------------------------------
Candidate traffic percentage: 10%
Selected model: production
Prediction: ENGINEERING
Confidence: 34.00%
Latency: 107.64 ms

Results saved to: C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2\artifacts\production\canary_release_results.c
sv

(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>python -m app.ml.production_experiments
```

Figure 20:Canary Release Experiment Results

```
(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>python -m app.ml.production_experiments

Shadow Deployment Results
----------------------------------------------
User-facing production prediction: ENGINEERING
Production confidence: 34.00%
Production latency: 117.73 ms

Hidden candidate prediction: ENGINEERING
Candidate confidence: 39.43%
Candidate latency: 9.76 ms

Prediction agreement: True

Results saved to: C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2\artifacts\production\shadow_deployment_result
s.csv
```

Figure 21:A/B Testing Experiment Results

```
(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>python -m app.ml.production_experiments

A/B Testing Results
----------------------------------------------
Group A (Production)
Prediction : ENGINEERING
Confidence: 34.00%
Latency   : 106.48 ms

Group B (Candidate)
Prediction : ENGINEERING
Confidence: 39.43%
Latency   : 2.53 ms

Prediction Agreement: True

Results saved to: C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2\artifacts\production\ab_testing_results.csv

(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>
(resume-screening) C:\Users\Haridass\OneDrive\Desktop\resume-screening-se4ml-assignment2>
```

6.6 Security Consideration

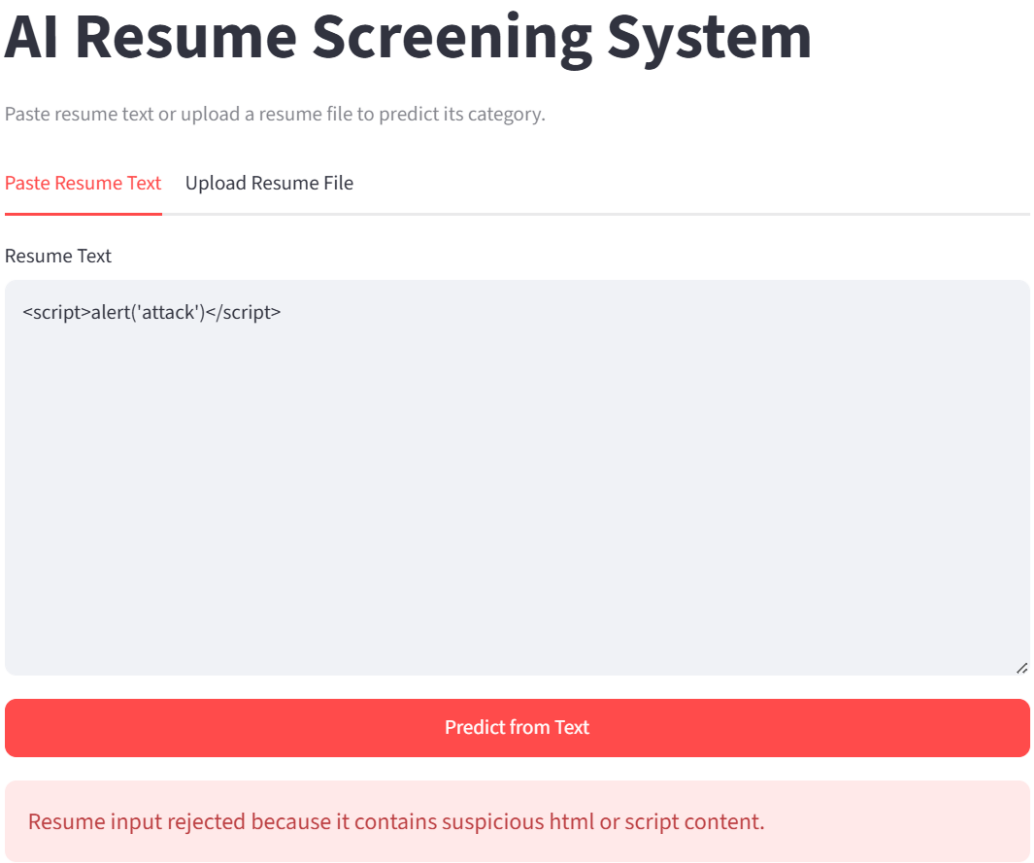
To improve the security and reliability of the application, input validation was implemented before processing prediction requests. The system validates user input and rejects suspicious or potentially malicious content before it reaches the machine learning model.

The implemented security validation includes:

- Validation of empty or insufficient resume text.
- Detection and rejection of suspicious HTML or JavaScript content.
- Validation of input parameters before model inference.
- Appropriate error messages for invalid requests.

These security measures help protect the application against malicious inputs and ensure that only valid resume data is processed by the machine learning model.

Figure 22:Security Validation in the Streamlit Application



7. Streamlit Application

A user-friendly web interface was developed using **Streamlit** to provide an interactive platform for resume classification. The application communicates with the FastAPI backend to perform machine learning inference and display prediction results in real time.

The Streamlit application provides the following features:

- Resume text input or file upload (PDF, DOCX, TXT).
- Real-time job category prediction.
- Prediction confidence and Top-3 predicted categories.
- Interactive visualization of prediction confidence.
- Model and data quality metrics dashboard.
- Quality assurance dashboard.
- Application log viewer.
- About page containing project and group information.

The interface enables users to interact with the machine learning model without requiring programming knowledge, providing a simple and intuitive user experience.

Figure 22. Streamlit User Interface for Resume Prediction

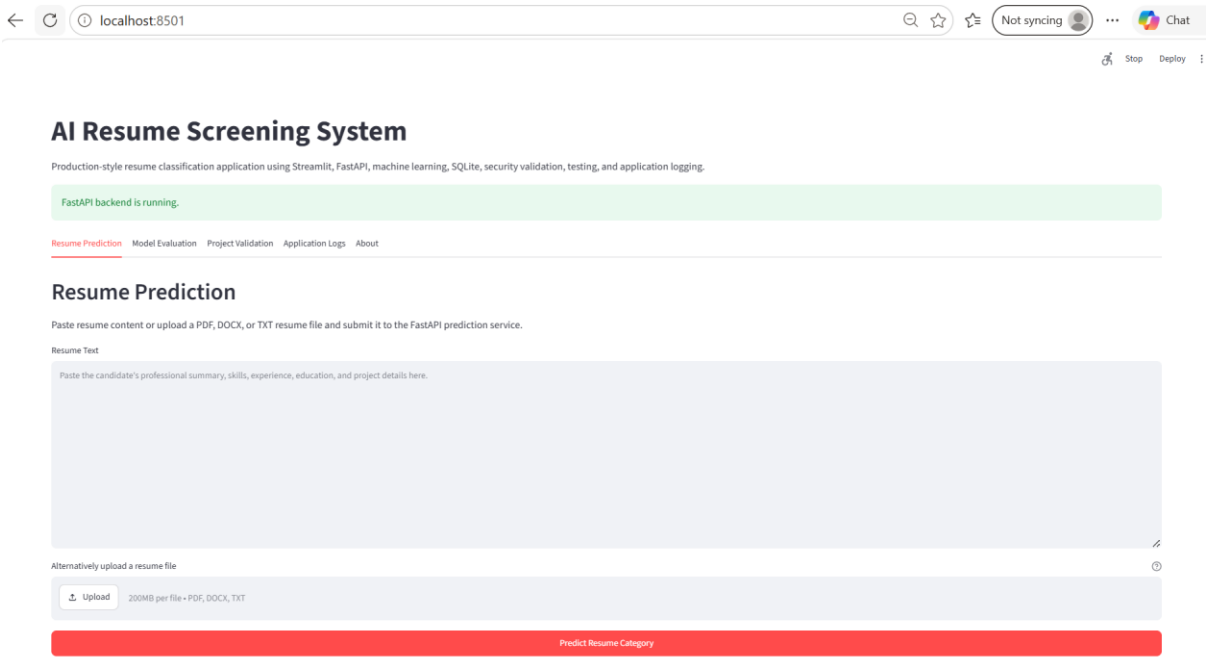


Figure 23. Resume Upload and Prediction Result in the Streamlit Application

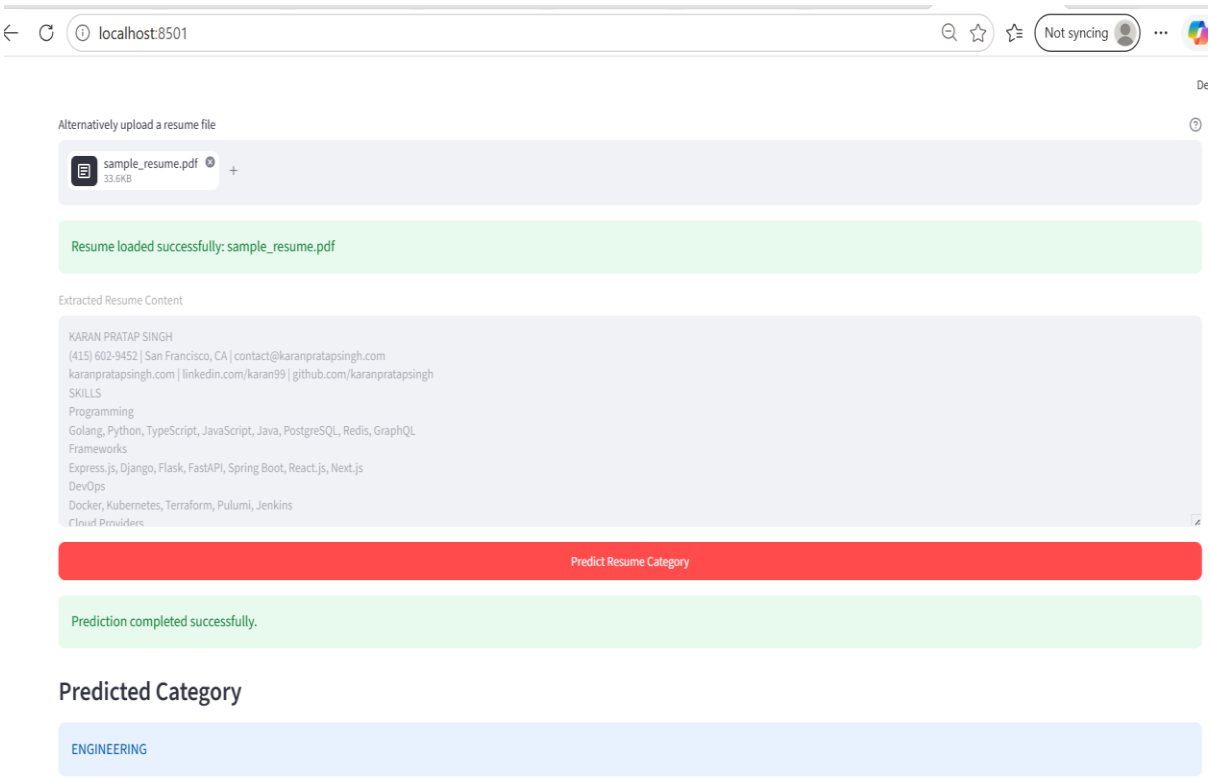


Figure 24. Model Evaluation Dashboard

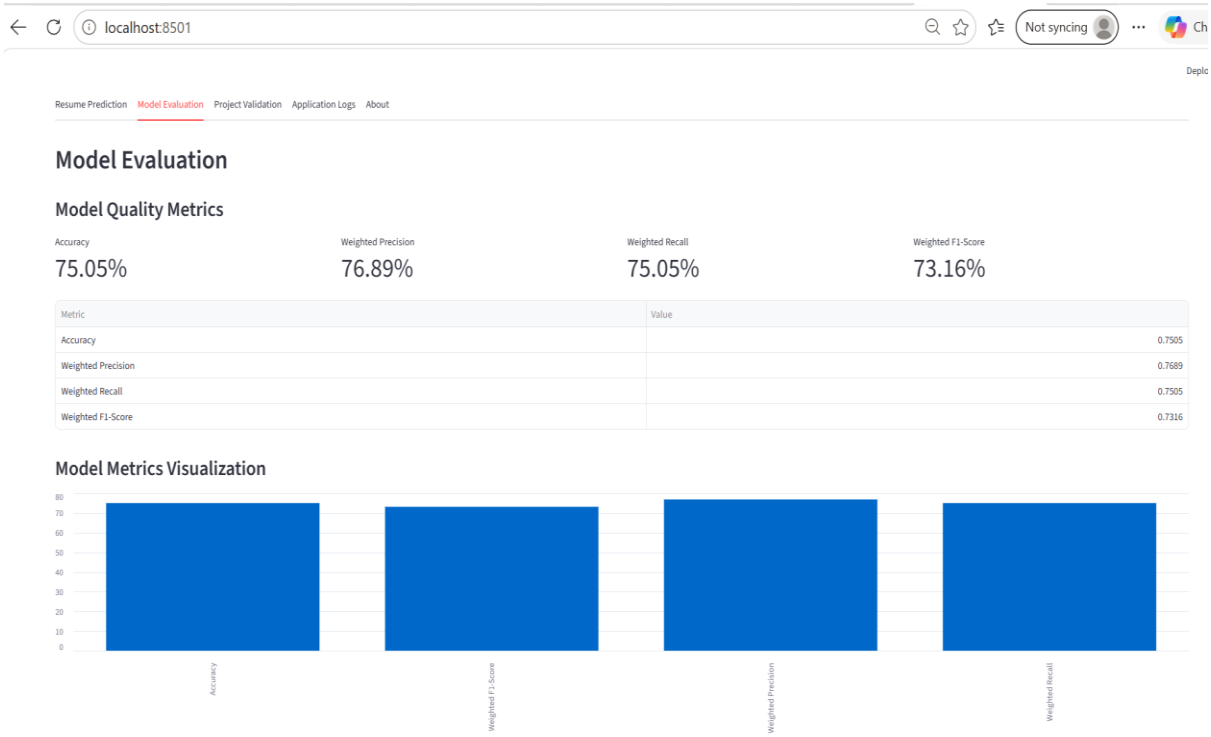


Figure 25. Data Quality Metrics Dashboard

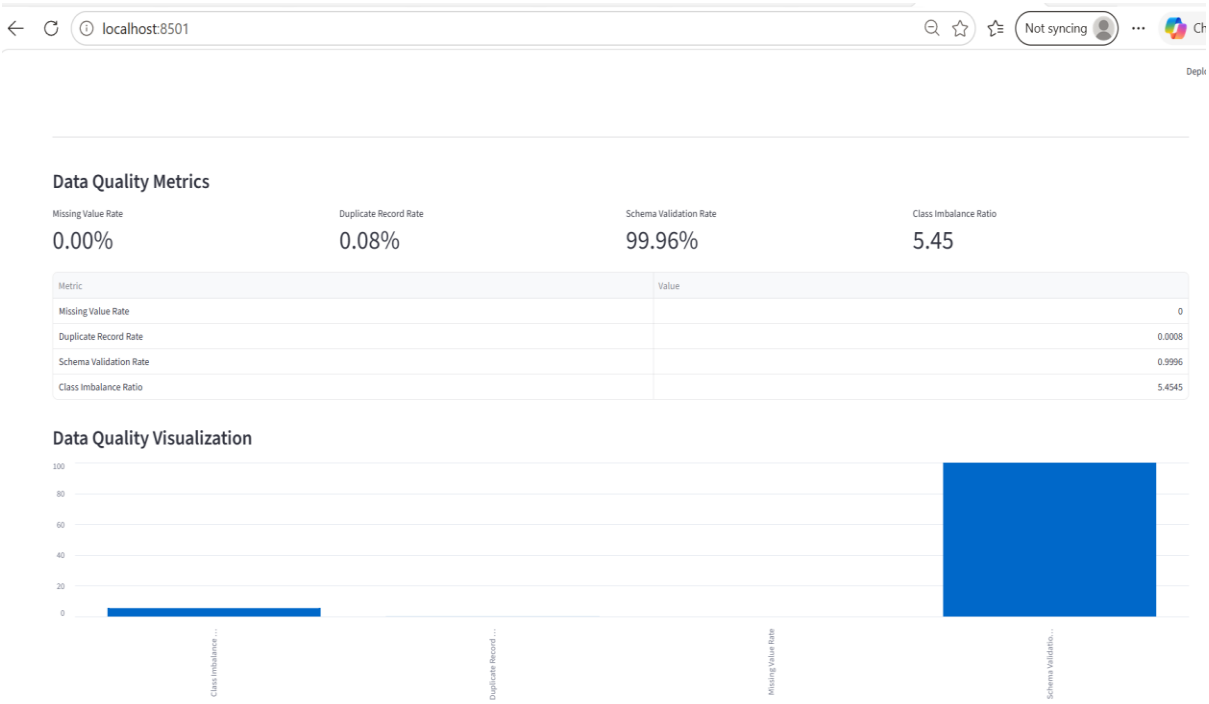


Figure 26. Project Validation Dashboard

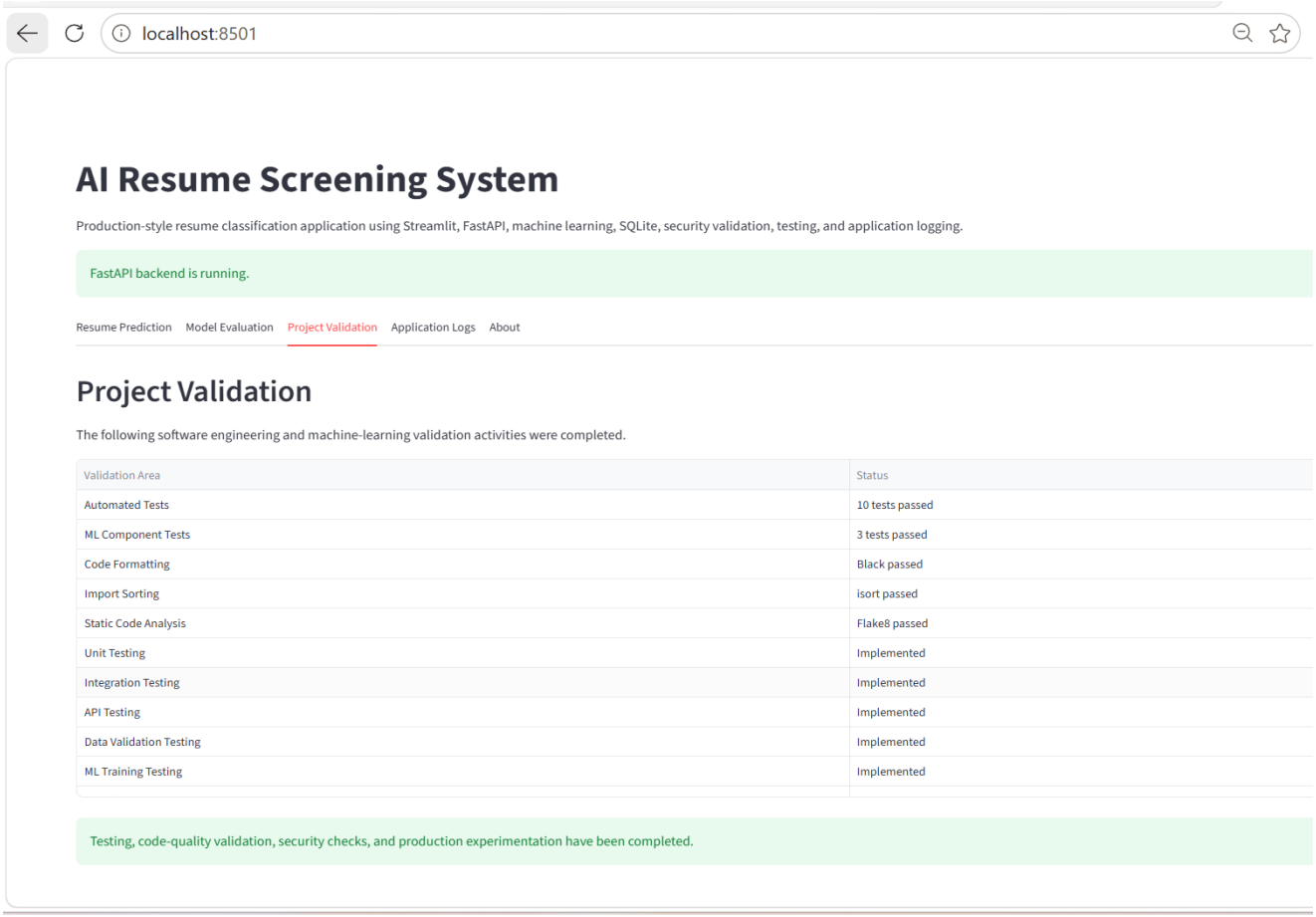


Figure 27. Application Logging Dashboard

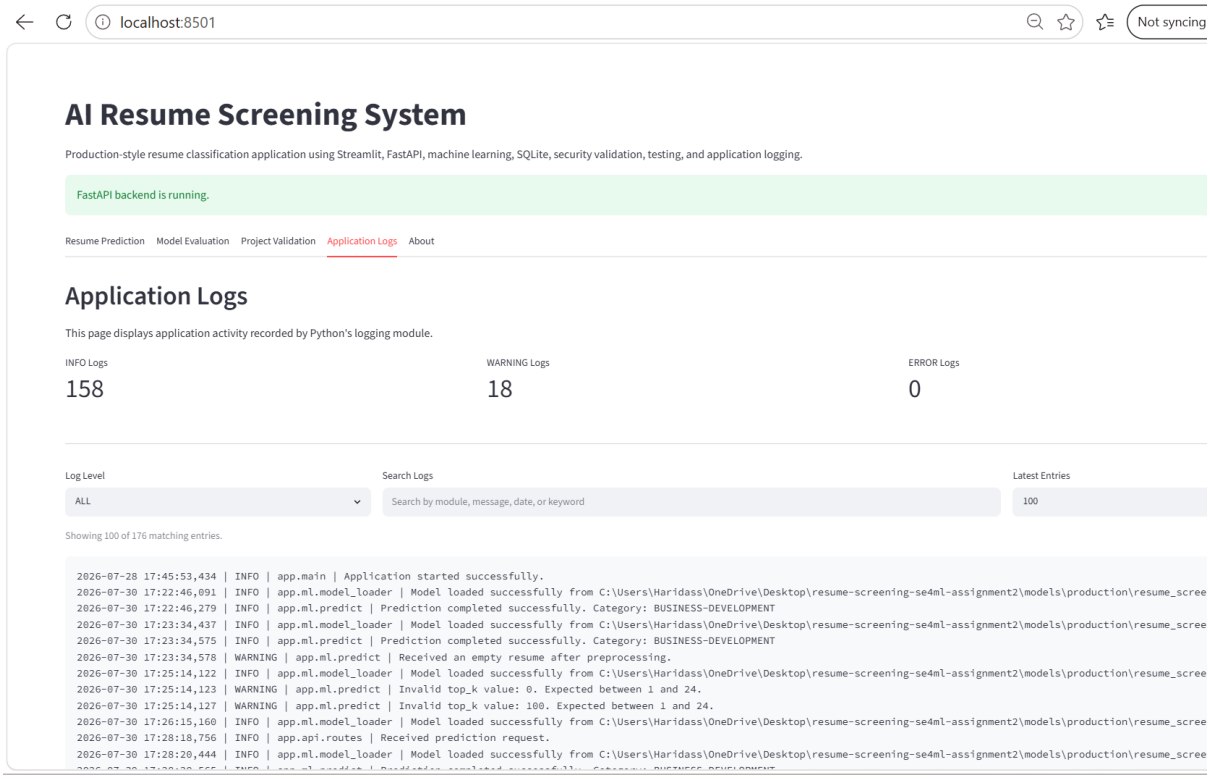
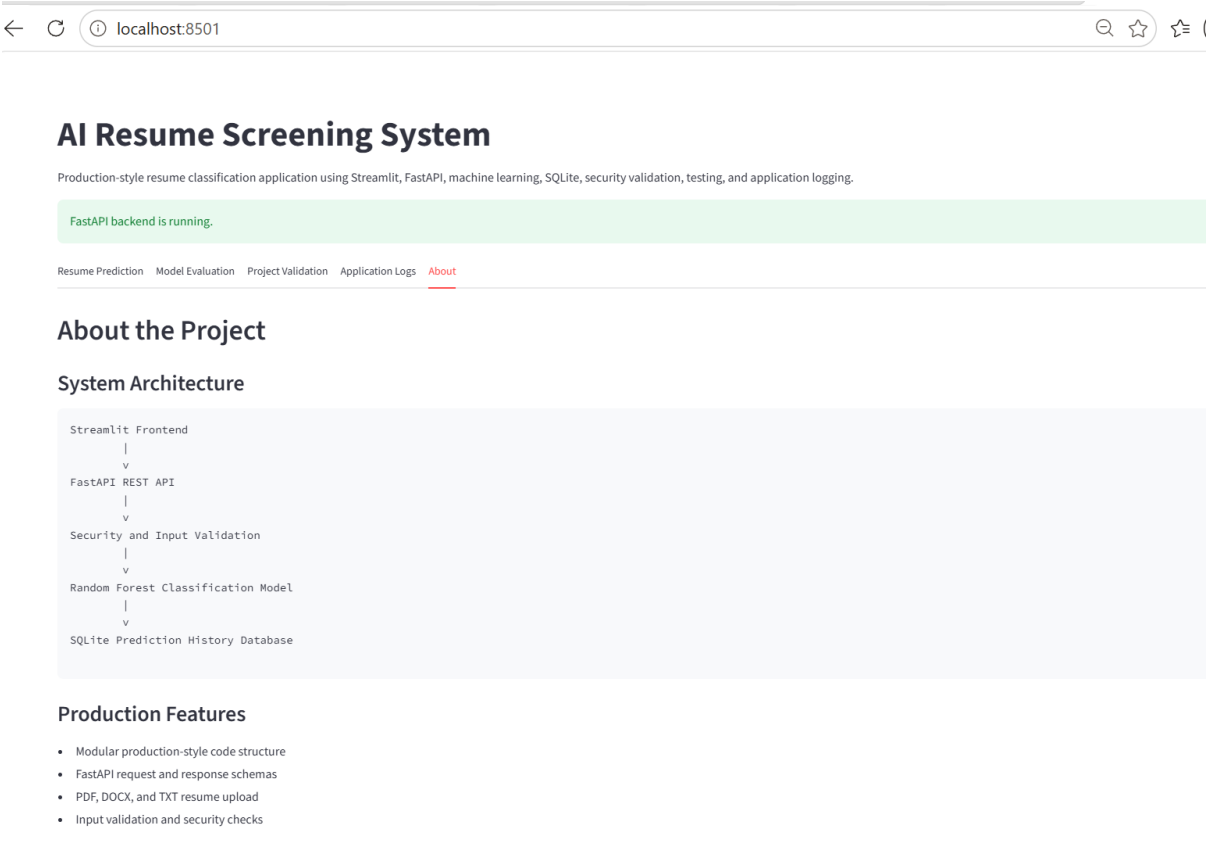


Figure 28. About Page of the Streamlit Application



8. Results and Discussion

The developed application successfully demonstrates the integration of machine learning with software engineering principles to build a production-ready resume screening system. The machine learning model was deployed through a FastAPI REST API and integrated with a Streamlit user interface for interactive prediction.

The application successfully incorporates modular software architecture, automated testing, logging, security validation, code quality verification, and production experimentation techniques. All implemented test cases passed successfully, and the model and data quality evaluations confirmed the reliability of the developed system.

Overall, the project satisfies the objectives of the assignment by demonstrating the transition from a research-oriented machine learning prototype to a maintainable and production-ready software application.

9. Conclusion

This project presented a Production-Ready AI Resume Screening and Job Role Prediction System developed using machine learning and software engineering best practices. The application extends the research prototype by incorporating modular software architecture, REST API implementation, automated testing, logging, security validation, code quality assessment, and production experimentation techniques.

The developed solution successfully meets the requirements of the Software Engineering for Machine Learning assignment while demonstrating how machine learning models can be deployed as reliable, maintainable, and scalable software applications.

10. Reproducibility

The project can be reproduced by following the steps below.

Prerequisites

The following software is required:

- Python 3.11

- Conda (Anaconda or Miniconda)
- Git

Step 1: Clone the Repository

git clone https://github.com/Haridass-K/resume-screening-se4ml-assignment2.git

Step 2: Create the Python Environment

conda create -n resume-screening python=3.11

conda activate resume-screening

Step 3: Install the Required Dependencies

pip install -r requirements.txt

Step 4: Download the Dataset

The resume dataset is not included in the repository because of GitHub and LMS file size limitations.

Download the dataset from:

https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset

Place the downloaded file in:

data/Resume.csv

Step 5: Verify the Project

Run the following commands to verify the project configuration:

black .

isort .

flake8 .

pytest -v

Step 6: Start the FastAPI Backend

uvicorn app.main:app --reload

The REST API documentation will be available at:

http://127.0.0.1:8000/docs

Step 7: Launch the Streamlit Application

Open a new terminal and execute:

streamlit run app/ui/streamlit_app.py

The application will be available at:

http://localhost:8501

Step 8: Reproduce the Research Phase

To reproduce the machine learning experiments and retrain the models, execute the research notebook:

notebooks/01_model_research.ipynb

The trained production model included in the repository can be used directly without retraining.

11. GitHub Repository

The complete project, including the source code, research notebook, trained model, documentation, and application files, is available in the following GitHub repository:

Repository:

https://github.com/Haridass-K/resume-screening-se4ml-assignment2